Übungen
zur Vorlesung
Betriebssysteme
Prof. Dr. Kornmayer

Übungen 3 – Prozess-Modelle und Threads

Aufgabe 3.1: Beschreiben sie die wesentlichen Unterschiede zwischen Programm und Prozess!

Aufgabe 3.1: UNIX Prozessverwaltung
Erzeugen Sie zwei Hintergrundprozesse. Jeder dieser Prozesse soll eine bestimmte Anzahl von Sekunden lang nichts tun (siehe sleep), dann eine Meldung ausgeben ("Hier Prozess 1" bzw. "Hier Prozess 2"), dann nochmals dieselbe Zeit schlafen und zum Abschluss nochmals die Meldung ausgeben.

- Schreiben Sie dazu ein Shellscript, das mit der Kennziffer und den Sekundenzahlen parametrisiert ist und das Sie zweimal aufrufen.
- Geben Sie den Prozessstatus des Systems aus, während die Prozesse schlafen (siehe pr)
- Beenden Sie einen Prozess vorzeitig (siehe kill)
- Starten Sie einen Hintergrundprozess, der seinerseits drei neue Hintergrundprozesse startet.

Jeder dieser drei Prozesse soll unmittelbar hintereinander 200-mal die Meldung „Hallo, hier ist Prozess x" ausgeben, wobei x je nach Prozess „A" „B" oder „C" ist. Auch hier lassen sich gut Shellscripts einsetzen.
Was beobachten Sie bezüglich der Reihenfolge der Prozessausführung?

Dokumentieren Sie die UNIX-Kommandos, mit denen Sie die Aufgaben gelöst haben.

Aufgabe 3.3: In der Vorlesung wurde das Datenmodell für Prozesse besprochen. Obwohl wir wissen, dass Betriebssysteme in C geschrieben sind, werden wir jetzt mit JAVA arbeiten. Sie sollen mit Java ein Modell für den Prozess-Kontroll-Block (PCB) erstellen. Dieses Modell soll in späteren Aufgaben erweitert werden und einfache Interaktionen simulieren.

- Reflektieren Sie nochmals das vorgestellte Datenmodell.
- Entwickeln Sie zuerst neben der Klasse ProcessControlBlock, die Klassen ProcessMetadata, ProcessThread, ProcessAddressSpace, ProcessFiles, die verschiedene Aspekte des PCB kapseln:
    - Achten Sie darauf, dass das Modell auch für mehrere Threads pro Prozess Gültigkeit haben kann. Entwickeln Sie im ersten Schritt aber ein Modell für 1 Thread pro Prozess.
    - Berücksichtigen Sie neben den Datenmodell auch das Zustandsmodell von Prozessen.
    - Gehen Sie von einer 64-Bit CPU aus, die insgesamt 16 Register hat. Die Inhalte dieser Register ist zu verwalten.
    - Für die Klasse ProcessAddressSpace implementieren eine leere Klasse als Platzhalter.
    - In der Klassen Processfiles verwenden Sie Platzhalter für StdInput, StdOutput, StdError und die Executable Datei.
- Implementieren Sie ein einfaches Beispiel, dass ein PCB mit Beispieldaten erstellt.

Aufgabe 3.4: Verwendung von fork()
Schreiben Sie unter C ein Programm, das die Funktionen fork verwendet, um ein Kindprozess zu erzeugen. Vater und Kind sollen jeweils in einer Schleife mit 500 Durchläufen jeweils den Schleifenzähler ausgeben. Lassen Sie das Programm mehrmals laufen und vergleichen Sie die Ausgaben! Erklären Sie das Verhalten des Programms.
(Es kann notwendig sein, dass Programm deutlich mehr als zweimal laufen zu lassen!)

Aufgabe 3.5: Verwenden Sie nun das JAVA Modell aus Aufgabe 2.4. Erweitern Sie die Implementierung um Prozesse einfach verwalten zu können.

- Reflektieren Sie nochmals die Bedeutung von Systemaufrufen/Systemcalls.
- Definieren Sie eine Schnittstelle IBSProcess mit den Funktionen
    - createProcess()
    - forkProcess()
    - killProcess()
    - (denken Sie auch über Funktionen nach, die Ihnen den aktuellen Zustand eines Systems ausgeben)
- Implementieren Sie eine Klasse BS und verwenden dazu die definierte Schnittstelle.
- Erzeugen Sie ein Main Programm, dass
    - zuerst 5 Prozesse exemplarisch erzeugt (createProcess()),
    - dann von diesen Prozessen mit forkProcess() jeweils ein Kind „kopiert“
    - und entfernen sie anschließend mit killProcess() die Prozesse mit der ID 5 und 8.

Aufgabe 3.6: Java Threads / Race Conditions

- Erstellen Sie eine Klasse, die als Container für zwei globale Variablen j und k dient (static). Initialisieren Sie die Werte j=0, k=5.
- Erstellen Sie eine Thread-Klasse Race, die in der run()-Methode 10-mal eine Schleife durchläuft, in der zuerst den Wert j gesetzt wird und anschließend die Operation k=k+j durchführt wird.
- Erzeugen Sie in der Main-Methode zwei Threads, die den Wert j auf 1 bzw. auf -1 setzen.

- Lassen Sie das Programm mehrmals laufen. Beschreiben Sie das Ergebnis!
- Erleichtern Sie nach jeder Operation innerhalb der Schleife einen Thread-Wechsel durch Verwendung von Thread.yield(). Wie ändert sich das Ergebnis?